Key

Style 4

- Truth and Nil 4
- Arithmetic 14
- Constants and Chance 36
- Number Theory 46
- Polynomials and Roots 53
- Matrices 60
- Arrays 68
- Boxes 83
- Sets and Membership 94
- Trains 103
- Functions 121
- Errors 130
- Classes and Properties 141
- The Virtual 151

Index 157

- English Index 158
- Glyph Index 162

Style

Contents

Truth and Nil 4

- Niladic 4
- Monadic 6
- Dyadic 8

Arithmetic 14

- Niladic 14
- Monadic 18
- Dyadic 29

Constants and Chance 36

- Niladic 36
- Monadic 40
- Dyadic 43

Number Theory 46

- Monadic 46
- Dyadic 49

Polynomials and Roots 53

- Monadic 53
- Dyadic 56

Matrices 60

- Niladic 60
- Monadic 62
- Dyadic 65

Arrays 68
Niladic 68
Monadic 71
Dyadic 78
Boxes 83
Niladic 83
Monadic 85
Dyadic 89
Sets and Membership 94
Niladic 94
Monadic 96
Dyadic 99
Trains 103
Niladic 103
Monadic 106
Dyadic 118
Functions 121
Monadic 121
Dyadic 126
Errors 130
Niladic 130
Monadic 136
Classes and Properties 141
Niladic 141
Monadic 144
Dyadic 147
The Virtual 151
Niladic 151
Monadic 154
Index 157

Truth and Nil: Niladic

Wuxia

俠

The Hero

U+4FA0

Nil



The empty set — quiet failure value; absorbs; Nil⊡function → Nil

U+2205

Truth

⊤

Verum — first-class truth; no true anymore

U+22A4

Truth and Nil: Monadic

Invert

~

Invert — monadic only (tilde). Scalar: ~ 0 → 1, ~ 1 → 0 (user wording to confirm). Unary scalar verbs operate on array back ranks (new 2026-10-11)

U+007E

Not

¬

Not — monadic boolean not: ¬ Truth yields Nil, ¬ Nil yields Truth (new 2026-10-14)

U+00AC

Congruent

≡

Ambivalent: monadic binds train to lhs⊡function

U+2261

Truth and Nil: Dyadic

Less Than



Less Than — dyadic < A B yields Truth when A < B, Nil otherwise (new 2026-10-14)

U+003C

Greater Than



Greater Than — dyadic > A B yields Truth when A > B, Nil otherwise (new 2026-10-14)

U+003E

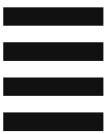
Congruent



Ambivalent: dyadic ≡ A B yields Truth when A and B have the same binding (same location in the system tables), Nil otherwise

U+2261

IsAt



IsAt — dyadic ≣ X Y yields Truth when X and Y share the same pointer address, Nil otherwise (new 2026-10-14)

U+2263

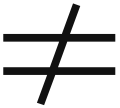
Equality



Equality — dyadic = A B yields Truth when A = B, Nil otherwise (new 2026-10-14)

U+003D

Unequal



Unequal — dyadic ≠ A B: sugar for ¬ = A B (boolean Not of Equality) (new 2026-10-14)

U+2260

Less Or Equal



Less Or Equal — dyadic ≤ A B yields Truth when A ≤ B, Nil otherwise (new 2026-10-14)

U+2264

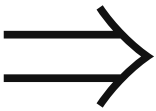
Greater Or Equal



Greater Or Equal — dyadic ≥ A B yields Truth when A ≥ B, Nil otherwise (new 2026-10-14)

U+2265

Implies



Implies — logical implication; ∅ ⇒ ∅ → ⊤

U+21D2

forall



Train quantifier — every element must evaluate non-nil. Idiom: dyadic NAND is ¬ ∀ 𝕀𝕀 X Y

U+2200

Approx



Approx — dyadic ≈ A B yields Truth when|A - B|is within 1e-6, Nil otherwise (new 2026-10-14)

U+2248

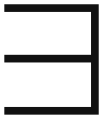
UnApprox



UnApprox — dyadic ≉ A B: sugar for ¬ ≈ A B (boolean Not of Approx) (new 2026-10-14)

U+2249

Forsome



Existential — at least one element evaluates non-nil

U+2203

Fornone



Not-forsome — defined as ~Forsome

U+2204

Arithmetic: Niladic

Signum / Plus



Ambivalent: Niladic + yields 0.
(Was ∓ U+2213, retired 2026-10-12

U+002B

Negate / Subtract

Niladic - yields 0 (new 2026-10-11

U+002D

Plus (Complex)



Plus (Complex), aka Sum Complex, Add Complex —; Niladic: sugar for ⨊ 0 0, the complex zero (new 2026-10-14)

U+2A22

Times (Complex)



Times (Complex) —; Niladic: sugar for ⨊ 1 0, the complex one (new 2026-10-14)

U+2A30

Minus (Complex)



Minus (Complex) —; Niladic: sugar for ⨌ 0 0, the complex zero (new 2026-10-14)

U+235B

Times (Polar)



Times (Polar) —; Niladic: the polar one, 2×1 [1; 0] (new 2026-10-14)

U+2A02

Divide (Complex)



Divide (Complex) —; Niladic: sugar for ⨌ 1 0, the complex one (new 2026-10-14)

U+2A2C

Divide (Polar)



Divide (Polar) —; Niladic: the polar one, 2×1 [1; 0] (new 2026-10-14)

U+2A38

Factorial

!

Factorial —; Niladic ! yields 1 (new 2026-10-12)

U+0021

Percent

%

Percent —; Niladic % is 0 (new 2026-10-12

U+0025

Square



Square — ⊠ X is X squared.; Niladic ⊠ yields 1, derived from Multiply (new 2026-10-12)

U+22A0

Multiply



*Multiply —; Niladic × yields empty array. Replaces Star * U+002A (new 2026-10-12; boxed forms 2026-10-12*

U+2A2F

Arithmetic: Monadic

Signum

+

Ambivalent: monadic Signum (less than zero yields −1, greater than zero yields 1, zero yields 0; back-rank scalar unary)

U+002B

Negate

-

Negate (monadic - X, unary minus)

U+002D

Conjugate



Conjugate — monadic ⊶ X: complex conjugate; negates the imaginary part, preserving orientation (on [r,i] yields [r,−i]; on [r;i] yields [r;−i]) (new 2026-10-15)

U+22B6

Double

+2

Double — monadic +⁼ X yields 2 × X (new 2026-10-15)

U+2A27

Sum



Sum — sugar for reduce-addition: ∑ X means / + X (redefined 2026-10-12)

U+2211

Square Root



Square Root — monadic √ X yields the square root of X (new 2026-10-15)

U+221A

Plus (Complex)



Plus (Complex), aka Sum Complex, Add Complex —; Monadic: if X is scalar, sugar for ℼ X 0 (promote to complex); else identity, yields X.

U+2A22

Times (Complex)



Times (Complex) —; Monadic: if X is scalar, sugar for ℼ X 0 (promote to complex); else identity, yields X.

U+2A30

Minus (Complex)



Minus (Complex) —; Monadic: if X is scalar, sugar for ℚ X 0 (promote to complex); else identity, yields X.

U+235B

Times (Polar)



Times (Polar) —; Monadic: if X is scalar, promote to polar 2×1; else identity, yields X.

U+2A02

Divide (Complex)



Divide (Complex) —; Monadic: if X is scalar, sugar for ℚ X 0 (promote to complex); else identity, yields X.

U+2A2C

Divide (Polar)



Divide (Polar) —; Monadic: if X is scalar, promote to polar 2×1; else identity, yields X.

U+2A38

Percent

%

Percent —; monadic % X is absolute value, a back-rank operator.; monadic redefined 2026-10-12)

U+0025

Star

*

*Star — exponentiate: monadic * X is 1, because the missing exponent Y is assumed 0, giving X^0 (new 2026-10-12). Idiom: X-th root of Y is * Y ÷ 1 X*

U+002A

Quotient

÷

Quotient —; Monadic ÷ X: if X is a square array, yields the matrix inverse; else reciprocal, a back-rank scalar operator (new 2026-10-12)

U+00F7

Multiply

×

Multiply —; Monadic × X yields X when X numeric; on boxed X, catenates once.; monadic redefined 2026-10-12)

U+2A2F

Ceiling



Ceiling —; monadic ⌈ X rounds up
(from APL; new 2026-10-12)

U+2308

Floor



Floor —; monadic ⌊ X rounds down
(from APL; new 2026-10-12)

U+230A

Sine



Sine (new 2026-10-12)

U+25DD

Cosine



Cosine (new 2026-10-12)

U+25DE

Arccosine



Arccosine (new 2026-10-12)

U+25F6

Arcsine



Arcsine (new 2026-10-12)

U+25F7

Tangent



Tangent (new 2026-10-12)

U+2AD7

Arctangent



Arctangent (new 2026-10-12)

U+2AD8

Hyperbolic Sine



Hyperbolic Sine (new 2026-10-12)

U+2AA6

Hyperbolic Cosine



Hyperbolic Cosine (new 2026-10-12)

U+2AA7

Hyperbolic Arcsine



Hyperbolic Arcsine (new 2026-10-12)

U+2ABD

Hyperbolic Tangent



Hyperbolic Tangent (new 2026-10-12)

U+2AD1

Natural Log



Natural Log —; monadic ⍦ X is ln X.; Monadic ⍦ on a polar 2×1 yields the rectangular complex pair [ln r, θ].

U+237C

Antilog



Antilog — sugar for Star: ⍋̇ X is e raised to the X. Monadic ⍋̇ on a rectangular complex pair yields the polar 2×1 [e^a; b] (new 2026-10-12; glyph was 𝒆 U+1D58A until 2026-10-14)

U+29CA

Hyperbolic Arctangent



Hyperbolic Arctangent (new 2026-10-12)

U+2A4D

Hyperbolic Arccosine



Hyperbolic Arccosine (new 2026-10-12)

U+2ABE

Factorial

!

Factorial —; monadic ! X is X factorial

U+0021

Fourier



Fourier — monadic F X: FFT of X (new 2026-10-15)

U+03DC

InverseFourier



InverseFourier — monadic ⅎ X: inverse FFT of X (new 2026-10-15)

U+2132

Base Ten Log



Base Ten Log — ⍋ X is log base 10 of X, sugar for ⍦ 10 X (new 2026-10-12)

U+2A5F

Magnitude



Magnitude — monadic ⍫X: absolute value of X (new 2026-10-16)

U+2357

Arithmetic: Dyadic

Plus



Ambivalent: dyadic + X Y yields X + Y (addition).; dyadic 2026-10-15)

U+002B

Subtract

Subtract (dyadic - X Y yields X - Y, first minus second), the subtract that works with the / right-fold Reduce.; dyadic 2026-10-12)

U+002D

Plus (Complex)



Plus (Complex), aka Sum Complex, Add Complex —; dyadic: sum of complex numbers (1×2 rectangular), per the structural complex convention; if either or both X and Y are scalar, promote via ℿ before adding.

U+2A22

Times (Complex)



Times (Complex) —; dyadic: product of complex numbers, per the structural complex convention; if either or both X and Y are scalar, promote via ℿ before multiplying.

U+2A30

Minus (Complex)



Minus (Complex) —; dyadic: difference of complex numbers, per the structural complex convention; if either or both X and Y are scalar, promote via ℿ before subtracting.

U+235B

Times (Polar)



Times (Polar) —; dyadic: product of polar numbers; uses the 2×1 oriented vector to hold magnitude and direction: magnitudes multiply, angles add; if either or both X and Y are scalar, promote to polar 2×1 before multiplying.

U+2A02

Divide (Complex)



Divide (Complex) —; dyadic: quotient of complex numbers, per the structural complex convention; if either or both X and Y are scalar, promote via ℿ before dividing.

U+2A2C

Divide (Polar)



Divide (Polar) —; dyadic: quotient of polar numbers; uses the 2×1 oriented vector to hold magnitude and direction: magnitudes divide, angles subtract; if either or both X and Y are scalar, promote to polar 2×1 before dividing.

U+2A38

Percent

%

Percent —; dyadic % X Y yields the remainder of X divided by Y

U+0025

Star

*

*Star — exponentiate: dyadic * X Y is X to the power Y (left=base, right=exponent)*

U+002A

Quotient

÷

Quotient —; dyadic ÷ X Y divides X by Y, yielding a real; division by zero yields infinity, except 0/0 which yields Nil.

U+00F7

Multiply

×

Multiply —; dyadic multiplies its arguments; when X is boxed, × X Y catenates X repeated Y times.

U+2A2F

Factorial

!

Factorial —; dyadic ! X Y is the binomial coefficient Y choose X.

U+0021

Ceiling

⌈

Ceiling —; dyadic ⌈ X Y is maximum

U+2308

Floor

⌊

Floor —; dyadic ⌊ X Y is minimum

U+230A

Natural Log



Natural Log —; Dyadic ⍟ X Y is log base X of Y, sugar for ÷ ⍟ Y ⍟ X (new 2026-10-12)

U+237C

Square



Square — ⊠ X is X squared.; Dyadic ⊠ X Y squares X Y times.

U+22A0

Constants and Chance: Niladic

Half

½

Half —; niladic: 0

U+00BD

Pi

π

Pi — niladic π: value of pi (new 2026-10-14)

U+03C0

Bell

𝔅

Bell —; niladic: random point in (-1, 1), exclusive

U+1D56D

Real

ℜ

Real — niladic: random real number between 0 and 1; sugar for ½ + 1 𝔅 (new 2026-10-13)

U+1D57D

Gamma



Gamma — niladic γ: Euler-Mascheroni constant (new 2026-10-14)

U+03B3

Golden



Golden — niladic φ: golden mean (new 2026-10-14)

U+03C6

Euler



Euler — niladic ℇ: Euler's constant (new 2026-10-14)

U+2107

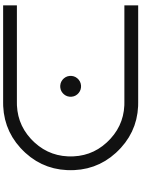
Dice



Dice —; niladic: single 6-sided die (1-6)

U+1D56F

Pick



Pick —; niladic: Nil

U+2A03

Constants and Chance: Monadic

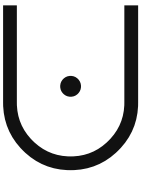
Half

½

Half —; monadic ½ X: X divided by 2; array → sugar for ⊞ (⍴ X); train → floor(length/2) index; dictionary → Nil (new 2026-10-13)

U+00BD

Pick



Pick —; monadic ⌣⊙X: scalar → same scalar; train → use Dice of size train to select element; array → use Dice of sides from shape to select element; dictionary → random {key:value} pair (new 2026-10-13)

U+2A03

Bell

𝔅

Bell —; monadic 𝔅 μ: bell curve point with mean μ (μ in (-1,1))

U+1D56D

Dice

𝔇

Dice —; monadic 𝔇 X: single X-sided die (1-X)

U+1D56F

Deal

?

Deal — given a scalar, works as expected; given an array, fills the entire back rank with limited values. Concrete examples and summary TBD (new 2026-10-11)

U+003F

Constants and Chance: Dyadic

Bell



Bell —; dyadic 𝔅 μ σ: bell curve point with mean μ and stddev σ (σ ≤ 1/3) (new 2026-10-13)

U+1D56D

Dice



Dice —; dyadic 𝔇 X N: N X-sided dice; all sugar via 𝔑 (hence 𝔅) (new 2026-10-13)

U+1D56F

Number Theory: Monadic

Divisible



Divisible —; monadic ∣ X yields Truth when X is divisible by 2, Nil otherwise

U+23A2

Indivisible



Indivisible —; monadic ∤ X: sugar for ¬ ∣ X (the Odd test)

U+2AEE

Primes



Primes — monadic ℙ X yields the prime factors of X, per J's q:. (new 2026-10-18)

U+1D4AB

Prime



Prime — monadic 𝔓 N yields the Nth prime, per J's p: (zero-indexed: 𝔓 0 is 2). (new 2026-10-18)

U+1D513

Prime



Prime — monadic ⌽ X: sugar for ⍭⍘ X ⍳ 2 ⌊ √ X ⋮ ≤ X 1 (new 2026-10-15)

U+23C0

Factor



Factor — factorization verb (new 2026-10-16)

U+27D2

Combinatoric



Combinatoric —; monadic ## X: factorial of X

U+2A69

Number Theory: Dyadic

LCM



LCM — dyadic ⨶ X Y: least common multiple of X and Y (new 2026-10-16)

U+238A

Divisible



Divisible —; dyadic ∣ X Y yields Truth when X is divisible by Y, Nil otherwise (new 2026-10-15)

U+23A2

GCD



GCD — dyadic ⫻ X Y: greatest common divisor of X and Y (new 2026-10-16)

U+2980

Indivisible



Indivisible —; dyadic ⫲ X Y: sugar for ¬ ∣ X Y (boolean Not of Divisible) (new 2026-10-15)

U+2AEE

Combinatoric



Combinatoric —; dyadic ⋕ X Y: binomial X choose Y (new 2026-10-16)

U+2A69

Polynomials and Roots: Monadic

NextLocal



NextLocal — the Roots division step: given P, yields [R prime, P prime] where R prime is Local(P) and P prime is P deflated by synthetic division of R prime. (new 2026-10-19)

U+2113

Roots



Roots — sugar: ℞ P is ℜ ℓ P (Reveal NextLocal over P); yields all polynomial roots. (new 2026-10-18; factored via Reveal 2026-10-19)

U+211E

Differentiate



Differentiate — train verb ∂ T yields symbolically differentiated version of train T of coefficients, multiplying constants as needed; same order as Polynomial (new 2026-10-13)

U+2202

Integrate



Integrate — train verb ∫ T yields symbolically integrated version of train T of coefficients, dividing constants as needed; same order as Polynomial (new 2026-10-13)

U+222B

Solve



Solve — monadic & A: array → vector of polynomial coefficients; missing input row is all zeroes (new 2026-10-13)

U+0026

Optimize



*Optimize —; monadic 𝔅 V: vector → polynomial; searches free variable with delta = 0.01 * domain, epsilon defaults to 1e-6*

U+214B

Local



Local — monadic 𝔴 V is 𝔅 𝕀𝕀𝕀 ℒ & V (FixedPoint Optimize Triple Solve V); approximates best x for a local minimal of V (new 2026-10-13)

U+1D50F

Polynomials and Roots: Dyadic

Reveal



Reveal — intrinsic. Dyadic ℜ P Y yields the successive application of P upon Y as Y shrinks to Nil. Epsilon from P has two elements: an alpha to apply P to next Y, and the next Y. (new 2026-10-18)

U+211C

Optimize



Optimize —; dyadic ϐ V ε: recurse into smaller domains until change in c < ε (new 2026-10-13)

U+214B

Integer Division



Integer Division — dyadic ⫽ X Y divides X by Y, yielding an integer (replaces Slash / integer division from manual p.19; / is now Reduce). Vector form: ⫽ V X where V is vector yields synthetic quotient of V by X (new 2026-10-14)

U+2AFD

Evaluate



Evaluate — dyadic 𝒇 X T evaluates train T of coefficients upon X (new 2026-10-13)

U+1D58B

FixedPoint



Fixed point — dyadic cø V guess: refine guess until epsilon < 1e-6 (max 100 steps); failure to converge returns infinity (new 2026-10-13)

U+AB44

Matrices: Niladic

Outer Product



Outer Product —; Niladic ⊗ yields 1 (new 2026-10-12)

U+2297

Inner Product



Inner Product —; Niladic · yields 1 (new 2026-10-12; matrix case 2026-10-14)

U+22C5

Multiply Matrices



Multiply Matrices —; Niladic ⋈ yields the 1×1 identity matrix (new 2026-10-14)

U+22C8

Matrices: Monadic

Outer Product



Outer Product —; Monadic ⊗ X yields X.

U+2297

Inner Product



Inner Product —; Monadic · X yields X.

U+22C5

Multiply Matrices



Multiply Matrices —; Monadic ⋈ X yields X.

U+22C8

L Factor



L Factor — monadic: the L (lower triangular, 1s on diagonal) factor of the LU decomposition of a rank-2 array (new 2026-10-14)

U+1D577

Determinant



Determinant — monadic ⌹X: determinant of square matrix X (new 2026-10-16)

U+233C

U Factor



U Factor — monadic: the U (upper triangular) factor of the LU decomposition of a rank-2 array (new 2026-10-14)

U+1D580

Matrices: Dyadic

Outer Product



Outer Product —; dyadic ⊗ X Y yields the outer product matrix where entry [i,j] is × X[i] Y[j]; shape is the concatenation of the shapes of X and Y.

U+2297

Inner Product



Inner Product —; dyadic · X Y: for vectors X, Y, the dot product (sum of pairwise products); for matrices, the Frobenius inner product: a scalar, the sum of all pairwise products.

U+22C5

Multiply Matrices



Multiply Matrices —; dyadic ⋈ X Y: matrix multiplication, (⋈ X Y)[i;j] = · X[i;] (⍤ Y)[j;]; inner dimensions must agree, non-square fine.

U+22C8

Arrays: Niladic

Pee



Pee — Where⊡Pee yields Where [0, 1
(first two coordinates)

U+2119

Zee



Zee — the last position of a slice
(Where⊡Zee yields Where [-1)

U+2124

Exx



Exx — the zero position of a slice
(Where [0)

U+1D54F

Wye



Wye — the midposition of a slice

U+1D550

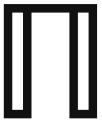
Extend



Extend —; niladic ⊏: sugar for ⊂ Nil

U+213E

Catenate



Catenate —; niladic ⊐: sugar for ⊂ Nil

U+213F

Grade Up



Grade Up — ⍋X yields the indices that sort X ascending. Niladic ⍋ yields 0 (from APL; new 2026-10-12)

U+234B

Grade Down



Grade Down — ⍒X yields the indices that sort X descending. Niladic ⍒ yields Nil (from APL; new 2026-10-12)

U+2352

Arrays: Monadic

Rank



Rank — monadic ⧺ X yields the number of dimensions in X; zero when scalar. (new 2026-10-18)

U+22D5

Permute



Permute —; monadic ⨋ X and dyadic ⨋ X Y work as in J's C.. Both modes intrinsic. (new 2026-10-19)

U+2A0A

Laminate



Laminate —; monadic ⩛ X: adds a leading singleton axis

U+2A5E

Anagram



Anagram — permutation index/parity (J's A.). Both modes intrinsic. (new 2026-10-19)

U+1D49C

Enumerate

№

Enumerate — monadic № X: sugar for ⍕⍳≢⍴X; yields index-value pairs for X (new 2026-10-18)

U+2116

Reflect



Reflect — monadic ÷X: sugar for ⍕⇋-⇋::X (new 2026-10-15)

U+223B

Count

74

⍳

U+2373

Kount



Kount — monadic ⍭B: takes a vector of T or Nil; returns a vector counting the T, turning each Nil to 0. Intrinsic. (new 2026-10-18)

U+2A23

⍳ Count

U+2373

Count — magical: when given a shape, fills an array matching shape. Scalar: behaves as normal. Array: fills array with next dimension limited by scalar from back rank — e.g. ⍳ 2 3 → rows of 0 1 2 (origin 0; last axis counts, outer axes replicate) (new 2026-10-11)

RavelRows



RavelRows — monadic ⍪ X: yields the rows of X as a vector (Antitable; new 2026-10-15)

U+22F2

Ravel



Ravel — monadic ⍎ X: flattens array X to vector, row-major (new 2026-10-14)

U+22F3

Reverse



Reverse —; monadic ⌽ X reverses elements along the last axis

U+233D

Transpose



Transpose — monadic ⍉ X reverses the order of axes (from APL; new 2026-10-12)

U+2349

Shape



Shape —; monadic ρ X: shape of X; scalar → 0; array → vector of dimensions; train → [1, length of train]; dictionary → [1, pairs present]

U+03C1

Extend



Extend —; monadic ⍮ X: Table X as in APL, adds trailing axis of length 1

U+213E

Catenate



Catenate —; monadic ⍪ X: catenates X to itself

U+213F

Rotate



Rotate —; monadic ⌽ X: rotate X toward origin by 1

U+29BD

Cardinality



Cardinality — "how big is this?"; scalar → -1; array → number of elements held; dictionary → number of pairs; train → length of train (new 2026-10-13)

U+2210

Center



Center — takes a shape, returns its integer halves expressed as a vector (e.g. [10, 6] → [5, 3]) (new 2026-10-13)

U+229E

Sight



Sight — takes 2D array, finds center, returns vector location of nearest non-zero scalar; ties pick one (new 2026-10-13)

U+2388

Arrays: Dyadic

Put



Put — ⫤⫥ X P A: value at coordinate P of array A becomes X (new 2026-10-18)

U+27DA

Permute



Permute —

U+2A0A

Laminate



Laminate —; dyadic ⩓ A B: yields [A; B]; when A and B are both scalars, yields A B (new 2026-10-17)

U+2A5E

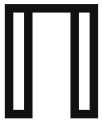
Anagram



Anagram — permutation index/parity (J's A.). Both modes intrinsic. (new 2026-10-19)

U+1D49C

Catenate



Catenate —; dyadic ⫿ X Y: catenate X and Y as in APL; variadic ⫿ A ... Z: catenate A through Z left to right (new 2026-10-14)

U+213F

Take



Take — ↑ X Y takes the first X elements of Y (last X if X is negative); X may be a vector for per-axis counts (from APL; new 2026-10-12)

U+2191

Drop



Drop — ↓ X Y drops the first X elements of Y (last X if X is negative); X may be a vector for per-axis counts (from APL; new 2026-10-12)

U+2193

Reverse



Reverse —; dyadic ⌽ X Y rotates Y by X positions (from APL; new 2026-10-12)

U+233D

Shape

ρ

Shape —; dyadic ρ S X: reshape X to shape S (per APL) (new 2026-10-13)

U+03C1

Extend



Extend —; dyadic ⍓ X N: extends by N axes (new 2026-10-14)

U+213E

Rotate



Rotate —; dyadic ⌽̂ X Y: rotate X by Y (Y positive toward origin, negative away) (new 2026-10-16)

U+29BD

Boxes: Niladic

Box



Box —; Niladic ⊂ yields Truth
(from APL Enclose; new 2026-10-12)

U+2282

Unbox



Unbox —; Niladic ⊃ yields Nil
(from APL Disclose; new 2026-10-12)

U+2283

Summarize



Summarize — the summary effect at full power. Array case: find the 0,0 corner; if another array, recurse into the subarray; if a scalar, yield the boxed scalar. Scalar case yields the boxed scalar; niladic ⊚ is the 1,1 vector containing zero.

U+229A

Boxes: Monadic

Box



Box — monadic ⧆ A: wraps A in a box so it travels as a single element (new 2026-10-17)

U+2256

Unbox



Unbox — monadic ⧈ A: opens the box, yielding its contents (new 2026-10-17)

U+2257

Catalogue



Catalogue —; monadic ⊚X: cartesian product of the boxed lists in vector X (new 2026-10-17)

U+2A37

Raze



Raze — monadic ⫻A: sugar for Ravel Map Unbox A; opens every box and joins the contents into one vector (new 2026-10-17)

U+2AF5

Box



Box —; monadic ⊂ X encloses X into a scalar box.

U+2282

Unbox



Unbox —; monadic ⊃ X yields X.

U+2283

Summarize



Summarize — the summary effect at full power. Array case: find the 0,0 corner; if another array, recurse into the subarray; if a scalar, yield the boxed scalar. Scalar case yields the boxed scalar

U+229A

Zip

88



U+22B0



Zip

`U+22B0`

Zip — takes a train of vectors and zips them: ⍭ V1 V2 ... Vn yields an m×n matrix (m = vector length, n = train length); the resulting width is the length of the train; a scalar in the train expands into a column to fit the others; a container in the train contributes only its origin scalar, recursively (the summary effect); mismatched lengths Nil-pad (consistent with Diction); zips along the back rank for higher-rank arrays; ⍭ 1 2 yields the column [1;2] (new 2026-10-12; recycles the retired AntiSignum glyph)

Boxes: Dyadic

Summarize

91



U+229A

Fetch



Fetch — dyadic ⊸ V X: applies V as search vector into X's items; follows path V down through nested boxes (new 2026-10-17)

U+22B8

Catalogue



Catalogue —; dyadic ⊗ A B: yields [[A0,B0] ... [A1,B1]] ... [[AN,B0] ... [AN,BN]], the outer product matrix of pairs

U+2A37

Link



Link — dyadic ⫢ X Y: sugar for Catenate Map Box X Y; boxes X and Y into a two-element list (new 2026-10-17)

U+2AE9



Summarize

U+229A

Summarize — the summary effect at full power. Array case: find the 0,0 corner; if another array, recurse into the subarray; if a scalar, yield the boxed scalar. Scalar case yields the boxed scalar; Dyadic D ⊚ A: D (Direction) is a rank-1 array of coordinates; summarization starts at A[D], then proceeds as monadic; a scalar Direction s means start at (s, 0); Zip summarizes each train element (new 2026-10-12)

Box



Box —; Dyadic ⊂ X Y encloses X by Y levels; Y=0 leaves X unchanged.

U+2282

Unbox

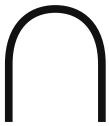


Unbox —; Dyadic ⊃ X Y picks element X from Y.

U+2283

Sets and Membership: Niladic

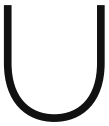
Intersection



Intersection —; niladic ∩: sugar for ⊂ Nil

U+2229

Union

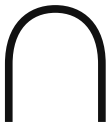


Union —; niladic ∪: sugar for ⊂ Nil

U+222A

Sets and Membership: Monadic

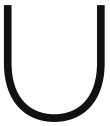
Intersection



Intersection —; monadic ∩ A: is A

U+2229

Union



Union —; monadic ∪ X: unique as in APL

U+222A

InOf



InOf — the curry of In: ⍸ InOf ⍷ ∈ X; monadic predicate testing membership in X. (new 2026-10-18)

U+22F8

Where



Where — monadic ⍸Y: returns vector of indices of all elements of array Y that are Truth (new 2026-10-18)

U+2A42

Enlist

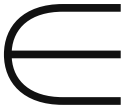


Enlist —; monadic ∈ Y: as defined in APL (deep flatten to vector)

U+220A

Sets and Membership: Dyadic

In



In — dyadic ∈ Y X yields Truth when X is in Y, Nil otherwise (J's e.). (new 2026-10-18)

U+2208

Bucket



Bucket — dyadic ∐ X Y: yields index of interval in X that Y falls into (new 2026-10-18)

U+2210

InLast



InLast — dyadic ∈̇ X Y: yields the index of the last occurrence of X in Y; yields length of Y if not found (new 2026-10-18)

U+22F5

InAt



InAt — dyadic ⋵ X Y: yields the index of Y whose value equals X; yields length of Y if not found (new 2026-10-18)

U+22F9

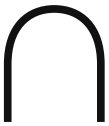
In



Enlist —; dyadic ∈ X Y: find/membership as defined in APL (new 2026-10-13)

U+220A

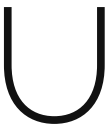
Intersection



Intersection —; dyadic ∩ A B: as in APL, elements in both (new 2026-10-14)

U+2229

Union



Union —; dyadic ∪ X Y: union as in APL (new 2026-10-14)

U+222A

Set Difference



Set Difference — dyadic ⍠A B: elements in A but not in B (new 2026-10-14)

U+2A43

Trains: Niladic

Reduce

/

Reduce — / F V: start at the highest index (its scalar is the initial result), step down toward origin; at each step result = F(scalar, result). Right fold.; Niladic / returns 0 (new 2026-10-12)

U+002F

Map



Map — ⟛ A B executes function A with zero arguments upon all scalars of B, as a back rank operator; usual back scalar rules.; Niladic ⟛ yields Truth, same as niladic 𝓘 (new 2026-10-12)

U+226C

Reduce (forward)

⌿

⌿ F V: start at the origin (its scalar is the initial result), step up toward the highest index; at each step result = F(result, scalar). Left fold, the reversed scan.

U+233F

Identity



Identity — 𝓘 V yields V unchanged; on a train, yields the train unchanged. Niladic 𝓘 yields Truth (new 2026-10-12)

U+1D574

Backstroke



Backstroke — Scan: the running Reduce, paired with /. \ F V on a length-n V yields a length-n vector where element i is / F (right fold) applied to the first i elements of V.; Niladic \ yields 0 (new 2026-10-12)

U+005C

Stop



Stop — statement separator; separates statements on one line

U+22C4

Backstroke (forward)



Backstroke (forward) — Reverse Scan, paired with ⍀. ⍀ F V on a length-n V yields a length-n vector where element i is ⍀F (left fold) applied to the first i elements of V.

U+2340

Strip



Strip —; niladic: removes the train from head up to but not including the first Stop encountered

U+2ACF

Trains: Monadic

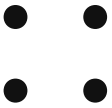
Meter

108



U+2214

Atomize



Atomize — monadic ∷ A: reads up to the first 32 elements of A (origin to end) and prepares them for use by train elements ahead (new 2026-10-14)

U+2237

Duplicate



Duplicate — copies a train to a train twice as long: ⊷ A B C → A B C A B C (was ==U+2A75, then ⁚ U+2236 — settled 2026-10-12)

U+22B7

Caboose



Caboose — picks up the train from head to Stop, then converts boxed contents to a vector (inverse of Atomize) (new 2026-10-15)

U+2A78



Meter

U+2214

Meter — measures its argument: ∔ V is ≺ + ⊷ V (zip the needle with the signal); yields M, a sequence of (needle, signal) pairs, needle ∈ -/0/+. M works in place of V in all Collate modes: the needle selects the function, the function transforms the signal (was ±, corrected to ≺ 2026-10-12)

Reduce



Reduce — / F V: start at the highest index (its scalar is the initial result), step down toward origin; at each step result = F(scalar, result). Right fold.; Monadic / V yields a linear array of the scalars ripped from V, highest to lowest indices.

U+002F

Map



Map — ⊘ A B executes function A with zero arguments upon all scalars of B, as a back rank operator; usual back scalar rules.; Monadic ⊘ B is identical to Identity 𝓘 B.

U+226C

Reduce (forward)



⌿ F V: start at the origin (its scalar is the initial result), step up toward the highest index; at each step result = F(result, scalar). Left fold, the reversed scan.; Monadic ⌿ V and niladic ⌿ behave identically to / (new 2026-10-12)

U+233F

Zero



Zero — on a train, replaces all values with zeroes while keeping shapes (new 2026-10-12)

U+1D585

Backstroke

111



U+005C

Duplicate Head



Duplicate Head — ⥣ A ... yields A A ...; duplicates head of train, aka duplicate/dup (new 2026-10-14)

U+2161

Backstroke (forward)



Backstroke (forward) — Reverse Scan, paired with ⫻. ⧹ F V on a length-n V yields a length-n vector where element i is ⫻F (left fold) applied to the first i elements of V.; Monadic ⧹ V and niladic ⧹ behave identically to \ (new 2026-10-12)

U+2340

Pipeline



Pipeline —

U+2AF7



Backstroke

`U+005C`

Backstroke — Scan: the running Reduce, paired with /. \ F V on a length-n V yields a length-n vector where element i is / F (right fold) applied to the first i elements of V.; Monadic \ V yields a linear array of the scalars ripped from V, lowest to highest indices.

Head



Head — sugar for ⊐1 (new 2026-10-15)

U+2991

Tail



Tail — sugar for ⊐-2 (new 2026-10-15)

U+2992

Duplicate Tail



Duplicate Tail — ⧟... A yields ... A A; duplicates tail of train (new 2026-10-13)

U+29DE

Strip

113



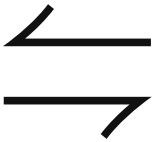
U+2ACF

⊐ Strip

U+2ACF

Strip —; monadic ⊐ X: removes the train from position X (0 is head) up to but not including the first Stop encountered; when X < 0, strips from position X in reverse order back to head, where −1 is the final tail position adjacent to the first Stop encountered (new 2026-10-15)

Swap



Swap — ⇋ A B ... yields B A ...; swaps head two of train. Single: ⇋ A Stop X yields X. Bare: ⇋ Stop X yields X (new 2026-10-13)

U+21CB

Replicate

115



U+232C

Rotate Head



Rotate Head — ⟲ A B C ... yields B C ... A; rotates head to tail (new 2026-10-13)

U+27F2

Rotate Tail



Rotate Tail — ⟳ ... A B C yields C ... A B; rotates tail to head (new 2026-10-13)

U+27F3



Replicate

U+232C

Replicate — ⌬ N X Stop yields X replicated N times; e.g. ⌬ 5 A Stop gives A A A A A. Zero case: ⌬ 0 A Stop X yields X. Negative N is antireplicate: ⌬ -1 A A A A A Stop yields A A A A. Empty antireplicate: ⌬ -1 Stop X yields X. Bare: ⌬ Stop X yields X (new 2026-10-13)

Nip



Nip — ↤ A B ... yields A ...; drops second, keeps head (new 2026-10-13)

U+21F7

Tuck



Tuck — ↦ A B ... yields A B A ...; copies head under second (new 2026-10-13)

U+21F8

Reverse Train



Reverse Train — reverses order of train (new 2026-10-13)

U+21F9

Over



Over — ⤺ A B ... yields B A B ...; copies second to head (new 2026-10-13)

U+293A

Triple

Ⅲ

Triple — Ⅲ X yields X X X; triplicates top of stack (new 2026-10-13)

U+2162

Fork



Fork — ⋔ F G H X: sugar for G H Swap F Dup X (J fork; new 2026-10-14)

U+22D4

Drop Train



Drop Train — sugar for reverse, replicate -1, reverse; drops head of train (new 2026-10-13)

U+291A

Trains: Dyadic

Applyself



Applyself — ⍣⍤1 f n a applies f n times to a; sugar for ⫷a ⍴⊂⍣⍤1 f n (from APL power; new 2026-10-12)

U+2363

Pipeline



Pipeline —; dyadic ⫷X Y applies vector Y, readable as a train of monadic functions, in sequence to X (new 2026-10-12)

U+2AF7

Functions: Monadic

Diction



Diction — dict-backed decision-table constructor (was Vectorize, renamed 2026-10-11). ¡ A B → {A:B}; ¡ A B C D → {A:B,C:D}; monadic ¡ (a b c) self-maps into a set. Membership sugar: X member Y → ⊤ (X ¿ (¡ Y))

U+00A1

Glyph



Glyph — the name for a single character value. Glyph ← ≡ ⌧: Glyph's function is the literalize train (via monadic ≡) — to make a Glyph, literalize

U+00AE

Decide

123



U+00BF

Function



Function — gets glyph del (∇, nabla): the function-definition operator. Monadic ∇ Y executes array Y as an FSA. (monadic 2026-10-19)

U+2207

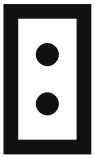


Decide

U+00BF

Decide — first-match generalized switch/match over n×2k train tables (was When, 2026-10-11). n×2 returns 1×1 (scalar); n×4 returns 1×2; generally n×2k returns 1×k. An explicit trailing C is unmatched-result filler, not an else branch; without filler, unmatched returns Nil without logging an error. Canonical: R ← X ¿ ¡ A B C

Print



Print — the quad-colon verb: prints the value from its right, then passes that value to its left (new 2026-10-11)

U+2360

(literalize)



Literalize next nonspace char, or ※0x???? by codepoint

U+2A33

Rail



Rail — monadic ⌐¯X yields a vector of symbols from train X (train-to-array conversion). (new 2026-10-19)

U+2ACD

Lay



Lay — monadic ¯¬V yields the train laid down from origin to limit of vector V (array-to-train, inverse of Rail). (new 2026-10-19)

U+2ACE

Lambda

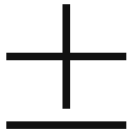


Lambda — lambda abstraction (new 2026-10-13)

U+039B

Functions: Dyadic

Collate



Collate — executes one of three trains upon a value: for less than zero, for zero, or for greater than zero (was AntiSignum ≺ U+22B0, renamed 2026-10-12; complement to signum)

U+00B1

Curry

128



U+1D49E

$\mathcal{C}$ Curry

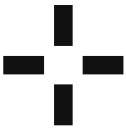
U+1D49E

Curry — intrinsic. Partial application: turns a dyadic verb into a monadic predicate by binding an argument, e.g. making = X from =. Semantics: Gets Q Curry P args: args to omega prime, omega prime shifts into alpha prime (binding the leftmost argument, textbook curry order), Q X makes X alpha, gets slice -1 of omega prime A, then call P; Q alpha yields P evaluated upon alpha prime, omega prime, and alpha as arguments in that order. Omega becomes nil upon Q's execution. Principle: Gets Q Curry P train is the principle for creating curries (Q is an example name assigned to the curried function, not a verb). (new 2026-10-18)

Errors: Niladic

Silence

132



U+22B9

Compress

133



U+22D6

Errline



Last error: which line

U+274C

Errmsg



Last error: what (type)

U+274E



Silence

U+22B9

Silence — ⊹ Z V drops all nonzero values; Z transforms the zero contents in sequence. ⊹ Z M on metered M: a nonzero needle drops the pair; a zero needle selects Z (applied to the signal).; Niladic ⊹ yields boxed Nil, a trial vector for enclosing operators to work on for starts (the dual of Compress; new 2026-10-12)



Compress

U+22D6

Compress — ⋖ P V applies transform P to each nonzero content of V; zero contents are eliminated; sequence order preserved. ⋖ P M on metered M: a nonzero needle selects P (applied to the signal); a zero needle drops the pair.; Niladic ⋖ yields boxed Truth, a trial vector for enclosing operators to work on for starts (a comprehension filter; new 2026-10-12)

Cut



Cut —; niladic: on error, erases the error and returns Nil

U+22EE

Debugger



✔ ⋄ *enters debugger iff error registers set*

U+2714

Errdmd



Last error: diamond position in line

U+2727

Succeed



Succeed — like Cut, except returns Truth (new 2026-10-15)

U+2999

Anticut



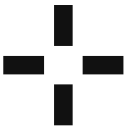
Anticut — on error throws to debugger; on return from debugger, returns Nil

U+22EF

Errors: Monadic

Silence

138



U+22B9

Compress

139



U+22D6

Cut



Cut —; monadic ⋮ X: checks X for Truth before returning

U+22EE



Silence

U+22B9

Silence — ⊹ Z V drops all nonzero values; Z transforms the zero contents in sequence. ⊹ Z M on metered M: a nonzero needle drops the pair; a zero needle selects Z (applied to the signal).; Monadic ⊹ V or ⊹ M means Z is identity: keep zeros (or zero-needle signals) as-is, drop nonzeros.



Compress

`U+22D6`

Compress — ⋖ P V applies transform P to each nonzero content of V; zero contents are eliminated; sequence order preserved. ⋖ P M on metered M: a nonzero needle selects P (applied to the signal); a zero needle drops the pair.; Monadic ⋖ V or ⋖ M means P is identity: pure filter, no transform.

Classes and Properties: Niladic

Constant

143

α

U+03B1

Epsilon

ε

Epsilon — holds the result of any getter, setter, or nabla when no explicit return glyph is used

U+03B5

Omega

ω

Omega — the vector of arguments passed to a setter (◅)

U+03C9

α Constant

`U+03B1`

Constant — declares a class constant: X α name ← value names a constant for class X. α is also the original (pre-override) value: read-only, may not be changed directly; trying registers an error. As the original value α is an ad hoc constant — contextual to the setter, not a declared class constant

Classes and Properties: Monadic

Increment



Increment —; monadic ⊕ D: invokes D's setter with nabla + 1 as the xthrement.

U+2295

Setter



Setter — the setter nabla: defines a property setter, as ∇ defines a function. Generalized: takes an xthrement argument — modifies instance data (increment/decrement generalized) rather than assigning (new 2026-10-14)

U+22B2

Getter



Getter — the getter nabla: defines a property getter, as ∇ defines a function

U+22B3

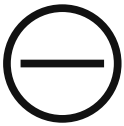
Property



Property —; Monadic ⋘ D: yields the result of D's getter (getter mode of D's class call) (new 2026-10-14)

U+22D8

Decrement



Decrement —; monadic ⊖ D: invokes D's setter with nabla - 1 as the xthrement.

U+2296

Divide By



Divide By —; monadic ⊘ D: invokes D's setter with nabla ÷ 2 as the xthrement (halve).

U+2298

Multiply By



Multiply By —; monadic ⊛ D: invokes D's setter with nabla × 2 as the xthrement (double).

U+229B

Classes and Properties: Dyadic

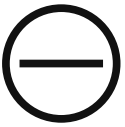
Increment



Increment —; Dyadic ⊕ D X: invokes D's setter with nabla + X as the xthrement (new 2026-10-14)

U+2295

Decrement



Decrement —; Dyadic ⊖ D X: invokes D's setter with nabla - X as the xthrement (new 2026-10-14)

U+2296

Multiply By



Multiply By —; Dyadic ⊛ D X: invokes D's setter with nabla × X as the xthrement (new 2026-10-14)

U+229B

Property



Property —; dyadic ⋘ I D: invokes D's getter (getter mode of D's class call), then invokes I's setter (setter mode of I's class call) with the result as argument; yields the assigned data as a pass-through.

U+22D8

Divide By



Divide By —; Dyadic ⊘ D X: invokes D's setter with nabla ÷ X as the xthrement (new 2026-10-14)

U+2298

(method binding)



Binds method to object: X⊡function

U+22A1

Gets



Gets — ⇔ A G: A stored in symbols[G], G stored in instances[A]; bidirectional binding (new 2026-10-13)

U+2B04

The Virtual: Niladic

IntegerBounded



IntegerBounded — bounded integer class: min/max enforced on creation, assignment, math; clamp + silent sticky flag on violation (new 2026-10-15)

U+012A

Trapdoor



Trapdoor — the assembly trap door: invokes machine code at a given address (installed via ctypes; proven 2026-10-14).

U+1D21D

VirtualSpace



VirtualSpace — the VR coordinate space: navel origin, forward half-sphere, [r;θ;φ], angles ±32767, distance in mm (new 2026-10-15)

U+1D4E5

Prim



Prim — base VR primitive: name, position, collision machinery; posted via Virtual (new 2026-10-16)

U+1D7D8

Raster



Raster — VR prim ⊑ Prim: array of pels, rank 4 [Y,X,1,3] (new 2026-10-16)

U+1D7D9

Point



Point — VR prim ⊑ Prim: dot prim, [N,6] (H,V,D,RGB) (new 2026-10-16)

U+1D7DA

Triangle



Triangle — VR prim ⊑ Prim: 9 points × 2 thetas + D + RGB = 22 numbers, [N,22] (new 2026-10-16)

U+1D7DB

The Virtual: Monadic

Trapdoor

156



U+1D21D

⊂ Trapdoor

`U+1D21D`

Trapdoor — the assembly trap door: invokes machine code at a given address (installed via ctypes; proven 2026-10-14).; Monadic ⊂ A: puts the address of A into the accumulator before calling. ⇔ X ⊂ ∇ assembly ∇: X gets the address of the compiled assembly script. If X is so bound, niladic X calls the code. ε maps to the accumulator. The call yields ε

Index

English Index

Anagram 72-4
Anagram 79-4
Anticut 135-1
Antilog 26-2
Applyself 119-1
Approx 11-3
Arccosine 24-1
Arcsine 24-2
Arctangent 24-4
Atomize 107-2
Backstroke 105-1
Backstroke 110-1
Backstroke (forward) 105-3
Backstroke (forward) 110-3
Base Ten Log 27-4
Bell 37-3
Bell 41-3
Bell 44-1
Box 84-1
Box 86-1
Box 87-1
Box 92-1
Bucket 100-2
Caboose 107-4
Cardinality 77-1
Catalogue 86-3
Catalogue 90-3
Catenate 70-2
Catenate 76-3
Catenate 80-1
Ceiling 23-1
Ceiling 33-2
Center 77-2
Collate 127-1
Combinatoric 48-3
Combinatoric 51-1
Compress 131-2
Compress 137-2
Congruent 7-3
Congruent 9-3
Conjugate 19-3
Constant 142-1
Cosine 23-4
Count 73-3
Curry 127-2
Cut 134-1
Cut 137-3
Deal 42-1
Debugger 134-2
Decide 122-3
Decrement 146-1
Decrement 148-2
Determinant 64-1
Dice 38-4
Dice 41-4
Dice 44-2
Diction 122-1
Differentiate 54-3
Divide (Complex) 16-3
Divide (Complex) 21-3
Divide (Complex) 31-3
Divide (Polar) 16-4
Divide (Polar) 21-4
Divide (Polar) 31-4
Divide By 146-2
Divide By 149-1
Divisible 47-1
Divisible 50-2
Double 19-4
Drop 80-3
Drop Train 117-3
Duplicate 107-3
Duplicate Head 110-2
Duplicate Tail 112-3
Enlist 98-1
Enumerate 73-1

Epsilon 142-2
Equality 10-1
Errdmd 134-3
Errline 131-3
Errmsg 131-4
Euler 38-3
Evaluate 57-4
Extend 70-1
Extend 76-2
Extend 81-2
Exx 69-3
Factor 48-2
Factorial 17-1
Factorial 27-1
Factorial 33-1
Fetch 90-2
FixedPoint 58-1
Floor 23-2
Floor 33-3
forall 11-2
Fork 117-2
Fornone 12-2
Forsome 12-1
Fourier 27-2
Function 122-4
Gamma 38-1
GCD 50-3
Gets 149-3
Getter 145-3
Glyph 122-2
Golden 38-2
Grade Down 70-4
Grade Up 70-3
Greater Or Equal 10-4
Greater Than 9-2
Half 37-1
Half 41-1
Head 112-1
Hyperbolic Arccosine 26-4
Hyperbolic Arcsine 25-3
Hyperbolic Arctangent 26-3
Hyperbolic Cosine 25-2
Hyperbolic Sine 25-1
Hyperbolic Tangent 25-4
Identity 104-4
Implies 11-1
In 100-1
In 101-1
InAt 100-4
Increment 145-1
Increment 148-1
Indivisible 47-2
Indivisible 50-4
InLast 100-3
Inner Product 61-2
Inner Product 63-2
Inner Product 66-2
InOf 97-3
Integer Division 57-3
IntegerBounded 152-1
Integrate 54-4
Intersection 95-1
Intersection 97-1
Intersection 101-2
InverseFourier 27-3
Invert 7-1
IsAt 9-4
Kount 73-4
L Factor 63-4
Lambda 125-1
Laminate 72-3
Laminate 79-3
Lay 124-4
LCM 50-1
Less Or Equal 10-3
Less Than 9-1
Link 90-4
(literalize) 124-2
Local 55-3
Magnitude 28-1
Map 104-2
Map 109-2
Meter 107-1
(method binding) 149-2

Minus (Complex) 16-1
Minus (Complex) 21-1
Minus (Complex) 31-1
Multiply 17-4
Multiply 22-4
Multiply 32-4
Multiply By 146-3
Multiply By 148-3
Multiply Matrices 61-3
Multiply Matrices 63-3
Multiply Matrices 66-3
Natural Log 26-1
Natural Log 33-4
Negate 19-2
Negate / Subtract 15-2
NextLocal 54-1
Nil 5-2
Nip 116-1
Not 7-2
Omega 142-3
Optimize 55-2
Optimize 57-2
Outer Product 61-1
Outer Product 63-1
Outer Product 66-1
Over 116-4
Pee 69-1
Percent 17-2
Percent 22-1
Percent 32-1
Permute 72-2
Permute 79-2
Pi 37-2
Pick 39-1
Pick 41-2
Pipeline 110-4
Pipeline 119-2
Plus 30-1
Plus (Complex) 15-3
Plus (Complex) 20-3
Plus (Complex) 30-3
Point 153-2
Prim 152-4
Prime 47-4
Prime 48-1
Primes 47-3
Print 124-1
Property 145-4
Property 148-4
Put 79-1
Quotient 22-3
Quotient 32-3
Rail 124-3
Rank 72-1
Raster 153-1
Ravel 75-2
RavelRows 75-1
Raze 86-4
Real 37-4
Reduce 104-1
Reduce 109-1
Reduce (forward) 104-3
Reduce (forward) 109-3
Reflect 73-2
Replicate 114-2
Reveal 57-1
Reverse 75-3
Reverse 80-4
Reverse Train 116-3
Roots 54-2
Rotate 76-4
Rotate 81-3
Rotate Head 114-3
Rotate Tail 114-4
Set Difference 101-4
Setter 145-2
Shape 76-1
Shape 81-1
Sight 77-3
Signum 19-1
Signum / Plus 15-1
Silence 131-1
Silence 137-1
Sine 23-3

Solve 55-1
Square 17-3
Square 34-1
Square Root 20-2
Star 22-2
Star 32-2
Stop 105-2
Strip 105-4
Strip 112-4
Subtract 30-2
Succeed 134-4
Sum 20-1
Summarize 84-3
Summarize 87-3
Summarize 90-1
Swap 114-1
Tail 112-2
Take 80-2
Tangent 24-3
Times (Complex) 15-4
Times (Complex) 20-4
Times (Complex) 30-4
Times (Polar) 16-2
Times (Polar) 21-2
Times (Polar) 31-2
Transpose 75-4
Trapdoor 152-2
Trapdoor 155-1
Triangle 153-3
Triple 117-1
Truth 5-3
Tuck 116-2
U Factor 64-2
UnApprox 11-4
Unbox 84-2
Unbox 86-2
Unbox 87-2
Unbox 92-2
Unequal 10-2
Union 95-2
Union 97-2
Union 101-3
VirtualSpace 152-3
Where 97-4
Wuxia 5-1
Wye 69-4
Zee 69-2
Zero 109-4
Zip 87-4

Glyph Index

!	(U+0021)	17-1, 27-1, 33-1	𝕣	(U+213E)	70-1, 76-2, 81-2
%	(U+0025)	17-2, 22-1, 32-1	ℼ	(U+213F)	70-2, 76-3, 80-1
&	(U+0026)	55-1	⅋	(U+214B)	55-2, 57-2
*	(U+002A)	22-2, 32-2	Ⅱ	(U+2161)	110-2
+	(U+002B)	15-1, 19-1, 30-1	Ⅲ	(U+2162)	117-1
-	(U+002D)	15-2, 19-2, 30-2	↑	(U+2191)	80-2
/	(U+002F)	104-1, 109-1	↓	(U+2193)	80-3
<	(U+003C)	9-1	⇋	(U+21CB)	114-1
=	(U+003D)	10-1	⇒	(U+21D2)	11-1
>	(U+003E)	9-2	↤	(U+21F7)	116-1
?	(U+003F)	42-1	↦	(U+21F8)	116-2
\	(U+005C)	105-1, 110-1	↤	(U+21F9)	116-3
~	(U+007E)	7-1	∀	(U+2200)	11-2
¡	(U+00A1)	122-1	∂	(U+2202)	54-3
¬	(U+00AC)	7-2	∃	(U+2203)	12-1
®	(U+00AE)	122-2	∄	(U+2204)	12-2
±	(U+00B1)	127-1	∅	(U+2205)	5-2
½	(U+00BD)	37-1, 41-1	∇	(U+2207)	122-4
¿	(U+00BF)	122-3	∈	(U+2208)	100-1
÷	(U+00F7)	22-3, 32-3	ϵ	(U+220A)	98-1, 101-1
Ī	(U+012A)	152-1	∐	(U+2210)	77-1, 100-2
Λ	(U+039B)	125-1	∑	(U+2211)	20-1
α	(U+03B1)	142-1	∔	(U+2214)	107-1
γ	(U+03B3)	38-1	√	(U+221A)	20-2
ε	(U+03B5)	142-2	∩	(U+2229)	95-1, 97-1, 101-2
π	(U+03C0)	37-2	∪	(U+222A)	95-2, 97-2, 101-3
ρ	(U+03C1)	76-1, 81-1	∫	(U+222B)	54-4
ϕ	(U+03C6)	38-2	∷	(U+2237)	107-2
ω	(U+03C9)	142-3	÷	(U+223B)	73-2
Ϝ	(U+03DC)	27-2	≈	(U+2248)	11-3
Ɛ	(U+2107)	38-3	≉	(U+2249)	11-4
ℓ	(U+2113)	54-1	≠	(U+2256)	86-1
№	(U+2116)	73-1	≟	(U+2257)	86-2
ℙ	(U+2119)	69-1	≠	(U+2260)	10-2
ℜ	(U+211C)	57-1	≡	(U+2261)	7-3, 9-3
℞	(U+211E)	54-2	≣	(U+2263)	9-4
ℤ	(U+2124)	69-2	⩽	(U+2264)	10-3
ⅎ	(U+2132)	27-3	⩾	(U+2265)	10-4

⊬	(U+226C)	104-2, 109-2	⍫	(U+234B)	70-3
⊂	(U+2282)	84-1, 87-1, 92-1	⍒	(U+2352)	70-4
⊃	(U+2283)	84-2, 87-2, 92-2	⍗	(U+2357)	28-1
⊕	(U+2295)	145-1, 148-1	⍛	(U+235B)	16-1, 21-1, 31-1
⊖	(U+2296)	146-1, 148-2	⍠	(U+2360)	124-1
⊗	(U+2297)	61-1, 63-1, 66-1	⍣	(U+2363)	119-1
⊘	(U+2298)	146-2, 149-1	⍳	(U+2373)	73-3
⊚	(U+229A)	84-3, 87-3, 90-1	⍷	(U+237C)	26-1, 33-4
⊛	(U+229B)	146-3, 148-3	⎈	(U+2388)	77-3
⊞	(U+229E)	77-2	⎊	(U+238A)	50-1
⊠	(U+22A0)	17-3, 34-1	⎢	(U+23A2)	47-1, 50-2
⊡	(U+22A1)	149-2	⏀	(U+23C0)	48-1
⊤	(U+22A4)	5-3	⏝	(U+25DD)	23-3
⊶	(U+22B0)	87-4	⏜	(U+25DE)	23-4
⊲	(U+22B2)	145-2	◶	(U+25F6)	24-1
⊳	(U+22B3)	145-3	◷	(U+25F7)	24-2
⊶	(U+22B6)	19-3	✔️	(U+2714)	134-2
⊷	(U+22B7)	107-3	✧	(U+2727)	134-3
⊸	(U+22B8)	90-2	❌	(U+274C)	131-3
⊹	(U+22B9)	131-1, 137-1	❎	(U+274E)	131-4
⋄	(U+22C4)	105-2	⟋	(U+27D2)	48-2
⋅	(U+22C5)	61-2, 63-2, 66-2	⟚	(U+27DA)	79-1
⋈	(U+22C8)	61-3, 63-3, 66-3	⟲	(U+27F2)	114-3
⋔	(U+22D4)	117-2	⟳	(U+27F3)	114-4
⋕	(U+22D5)	72-1	⟻	(U+291A)	117-3
⋖	(U+22D6)	131-2, 137-2	⤺	(U+293A)	116-4
⋘	(U+22D8)	145-4, 148-4	⦀	(U+2980)	50-3
⋮	(U+22EE)	134-1, 137-3	⦑	(U+2991)	112-1
⋯	(U+22EF)	135-1	⦒	(U+2992)	112-2
⋲	(U+22F2)	75-1	⦙	(U+2999)	134-4
⋳	(U+22F3)	75-2	⦼	(U+29BD)	76-4, 81-3
⋵	(U+22F5)	100-3	⧊	(U+29CA)	26-2
⋸	(U+22F8)	97-3	⧞	(U+29DE)	112-3
⋹	(U+22F9)	100-4	⨂	(U+2A02)	16-2, 21-2, 31-2
⌈	(U+2308)	23-1, 33-2	⨃	(U+2A03)	39-1, 41-2
⌊	(U+230A)	23-2, 33-3	⨊	(U+2A0A)	72-2, 79-2
⌬	(U+232C)	114-2	⨢	(U+2A22)	15-3, 20-3, 30-3
⍃	(U+233C)	64-1	⨣	(U+2A23)	73-4
⍎	(U+233D)	75-3, 80-4	⨧	(U+2A27)	19-4
⍏	(U+233F)	104-3, 109-3	⨪	(U+2A2C)	16-3, 21-3, 31-3
⍀	(U+2340)	105-3, 110-3	⨯	(U+2A2F)	17-4, 22-4, 32-4
⍉	(U+2349)	75-4	⨰	(U+2A30)	15-4, 20-4, 30-4

⨳	(U+2A33)	124-2
⨷	(U+2A37)	86-3, 90-3
⨸	(U+2A38)	16-4, 21-4, 31-4
⩅	(U+2A42)	97-4
⩃	(U+2A43)	101-4
⩍	(U+2A4D)	26-3
⩞	(U+2A5E)	72-3, 79-3
⩟	(U+2A5F)	27-4
⩩	(U+2A69)	48-3, 51-1
⩸	(U+2A78)	107-4
⪦	(U+2AA6)	25-1
⪧	(U+2AA7)	25-2
⪭	(U+2ABD)	25-3
⪮	(U+2ABE)	26-4
⫍	(U+2ACD)	124-3
⫎	(U+2ACE)	124-4
⫏	(U+2ACF)	105-4, 112-4
⫑	(U+2AD1)	25-4
⫖⊂	(U+2AD7)	24-3
⫗⊄	(U+2AD8)	24-4
⫩	(U+2AE9)	90-4
⫮	(U+2AEE)	47-2, 50-4
⫵	(U+2AF5)	86-4
⫷	(U+2AF7)	110-4, 119-2
⫽	(U+2AFD)	57-3
⇔	(U+2B04)	149-3
俠	(U+4FA0)	5-1
𫑄	(U+AB44)	58-1
𝆭	(U+1D21D)	152-2, 155-1
𝒜	(U+1D49C)	72-4, 79-4
𝒞	(U+1D49E)	127-2
𝒫	(U+1D4AB)	47-3
𝓥	(U+1D4E5)	152-3
𝔏	(U+1D50F)	55-3
𝔓	(U+1D513)	47-4
𝕏	(U+1D54F)	69-3
𝕐	(U+1D550)	69-4
𝔅	(U+1D56D)	37-3, 41-3, 44-1
𝔇	(U+1D56F)	38-4, 41-4, 44-2
𝔍	(U+1D574)	104-4
𝔏	(U+1D577)	63-4
𝔑	(U+1D57D)	37-4
𝔘	(U+1D580)	64-2
𝔷	(U+1D585)	109-4
𝖋	(U+1D58B)	57-4
𝟘	(U+1D7D8)	152-4
𝟙	(U+1D7D9)	153-1
𝟚	(U+1D7DA)	153-2
𝟛	(U+1D7DB)	153-3